

TP 4 : CI/CD et Déploiement AWS

Infrastructure as Code avec Terraform - Partie 2

Travail Pratique N°4



Saad KORCHI

Étudiant en Cybersécurité

Encadré par :

Pr. AMAMOU Ahmed

EIDIA Cybersécurité

SOMMAIRE

1	Introduction	3
2	Intégration Continue avec GitLab CI/CD	3
2.1	Structure du Dépôt GitLab	3
2.2	Définition de la Pipeline CI/CD	3
2.3	Gestion des Variables Sensibles	4
3	Authentification AWS depuis GitLab	4
4	Résolution des Problèmes Techniques	4
5	Déploiement et Accès à l'Application	4
5.1	Exécution de la Pipeline	4
5.2	Obtention de l'URL Publique	4
5.3	Validation	5
6	Bilan et Perspectives	5
6.1	Bilan du Projet	5
6.2	Perspectives d'Amélioration	5
	Captures d'écran	6

1 Introduction

Contexte - Partie 2

Cette seconde partie couvre l'intégration continue avec GitLab CI/CD, la gestion des variables sensibles, l'authentification AWS, la résolution des problèmes techniques et le déploiement final de l'application sur AWS Fargate.

2 Intégration Continue avec GitLab CI/CD

2.1 Structure du Dépôt GitLab

Le projet a été organisé de manière à séparer le code de l'application de celui de l'infrastructure :

```
1 upload-app-final/  
2     .gitlab-ci.yml      # Definition de la pipeline  
3     app/                # Code source Node.js  
4         server.js  
5         Dockerfile  
6     terraform/         # Code d'infrastructure  
7         main.tf
```

2.2 Définition de la Pipeline CI/CD

Le fichier .gitlab-ci.yml définit les différentes étapes :

```
1 stages:  
2   - validate  
3   - plan  
4   - apply  
5  
6 validate:  
7   stage: validate  
8   script:  
9     - gitlab-terraform validate  
10  
11 plan:  
12   stage: plan  
13   script:  
14     - gitlab-terraform plan  
15     - gitlab-terraform plan-json  
16  
17 apply:  
18   stage: apply  
19   script:  
20     - gitlab-terraform apply  
21   when: manual
```

2.3 Gestion des Variables Sensibles

Les clés d'accès AWS ont été stockées dans les variables CI/CD de GitLab :

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_DEFAULT_REGION

3 Authentification AWS depuis GitLab

La méthode retenue est l'utilisation des clés d'accès d'un utilisateur IAM. Les variables d'environnement définies dans GitLab sont automatiquement utilisées par le provider AWS de Terraform.

4 Résolution des Problèmes Techniques

Problèmes rencontrés et solutions

Problème	Solution
Virtualisation Docker	Activation de WSL 2 et Hyper-V sur Windows
Erreur "sh command not found"	Ajout de <code>entrypoint: [\""]</code> dans la configuration
Sous-module Git	Suppression du sous-module et réintégration standard
"duplicate required providers"	Fusion des blocs <code>required_providers</code>
Limite de VPCs atteinte	Nettoyage des VPCs inutilisés via console AWS
Erreur "stale plan"	Replanification avant <code>apply</code> dans le job

5 Déploiement et Accès à l'Application

5.1 Exécution de la Pipeline

Après les corrections, la pipeline GitLab s'est exécutée avec succès. Le job `apply` a été lancé manuellement.

5.2 Obtention de l'URL Publique

L'URL est le nom DNS fourni par l'ALB, récupérable via :

- La console AWS (EC2 → Load Balancers)
- Les sorties (outputs) de Terraform

Format de l'URL : `http://[nom-dns-alb].eu-west-3.elb.amazonaws.com`

5.3 Validation

Un test avec `curl` ou un navigateur web a confirmé que l'application répondait correctement.

6 Bilan et Perspectives

6.1 Bilan du Projet

Objectifs atteints

- Infrastructure définie par du code (IaC) avec Terraform
- Pipeline CI/CD fonctionnelle avec GitLab
- Application déployée et accessible publiquement sur AWS Fargate

6.2 Perspectives d'Amélioration

- **Sécurisation** : Ajout d'un certificat SSL (AWS Certificate Manager) et configuration HTTPS
- **Environnements multiples** : Pipelines distinctes pour dev, test, prod
- **Base de données** : Intégration d'Amazon RDS
- **Domaine personnalisé** : Association via Amazon Route 53
- **Observabilité** : Métriques et dashboards CloudWatch

Fin du rapport TP4

Captures d'écran



FIGURE 1 – Figure 6

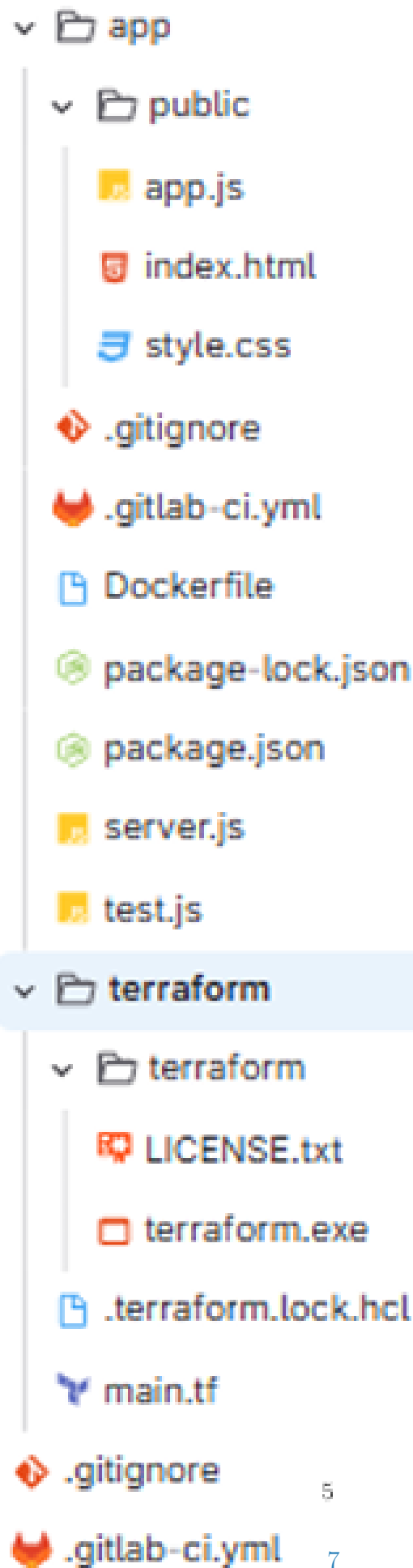


FIGURE 2 – Figure 7

```

#####
# Networking (single public vpc)
#####

resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name = "${var.app_name}-vpc"
  }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.app_name}-igw"
  }
}

resource "aws_subnet" "public_a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.0.0/24"
  availability_zone = data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.app_name}-public-a"
  }
}

resource "aws_subnet" "public_b" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.0.0/24"
  availability_zone = data.aws_availability_zones.available.names[1]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.app_name}-public-b"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "${var.app_name}-public-rt"
  }
}

resource "aws_route_table_association" "public_a" {
  subnet_id   = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_b" {
  subnet_id   = aws_subnet.public_b.id
  route_table_id = aws_route_table.public.id
}

```

FIGURE 3 – Figure 8


```
terraform {  
  backend "http" {  
  }  
}
```

FIGURE 4 – Figure 9

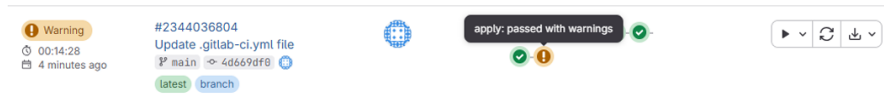


FIGURE 5 – Figure 10

CI/CD Variables </> 3				Reveal values		Add variable
Key ↑	Value	Environments	Actions			
AWS_ACCESS_KEY_ID		All (default)				
AWS_DEFAULT_REGION		All (default)				
AWS_SECRET_ACCESS_KEY		All (default)				

FIGURE 6 – Figure 11